

A Brief Primer on Zsh Commands

For macOS Terminal

Contents

1	Introduction	3
2	Navigation and File Management	3
2.1	Basic Navigation	3
2.2	File Operations	3
3	Viewing and Editing Files	3
4	Search and Find	4
5	Zsh-Specific Features	4
5.1	Advanced Globbing	4
5.2	Directory Stack	4
5.3	Tab Completion	4
6	Process Management	5
7	Permissions and Ownership	5
8	Input/Output Redirection	5
9	Variables and Environment	6
10	Aliases and Functions	6
10.1	Aliases	6
10.2	Functions	6
11	History and Shortcuts	7
11.1	Command History	7
11.2	Keyboard Shortcuts	7
12	Network and System	7
13	Package Management	8
14	Customizing Zsh	8
14.1	Configuration File	8
14.2	Oh My Zsh (Optional)	8
15	Quick Reference Table	9

16 Tips and Best Practices	9
17 Getting Help	9
18 Conclusion	10

1 Introduction

Zsh (Z shell) is the default shell on macOS since Catalina (10.15). It's backwards-compatible with bash while offering enhanced features like better tab completion, glob patterns, and themeable prompts.

To check your current shell:

```
echo $SHELL
```

2 Navigation and File Management

2.1 Basic Navigation

Command	Description
<code>pwd</code>	Print working directory
<code>ls</code>	List files and directories
<code>ls -la</code>	List all files (including hidden) with details
<code>cd ~</code>	Change to home directory
<code>cd ..</code>	Go up one directory
<code>cd -</code>	Go to previous directory

2.2 File Operations

```
# Create files and directories
touch filename.txt          # Create empty file
mkdir dirname               # Create directory
mkdir -p path/to/nested/dir # Create nested directories

# Copy, move, delete
cp file.txt copy.txt       # Copy file
cp -r dir/ newdir/         # Copy directory recursively
mv oldname.txt newname.txt # Rename/move file
rm file.txt                # Remove file
rm -r dirname/             # Remove directory recursively
rm -rf dirname/            # Force remove (use carefully!)
```

3 Viewing and Editing Files

Command	Description
<code>cat file.txt</code>	Display entire file
<code>less file.txt</code>	View file page by page (q to quit)
<code>head file.txt</code>	Show first 10 lines
<code>tail file.txt</code>	Show last 10 lines
<code>tail -f log.txt</code>	Follow file updates in real-time
<code>nano file.txt</code>	Edit file with nano editor
<code>vim file.txt</code>	Edit file with vim editor

4 Search and Find

```
# Find files
find . -name "*.txt"          # Find all .txt files
find . -type d -name "dirname" # Find directories

# Search within files
grep "search term" file.txt   # Search for text in file
grep -r "term" .              # Recursive search in directory
grep -i "term" file.txt       # Case-insensitive search
grep -n "term" file.txt       # Show line numbers
```

5 Zsh-Specific Features

5.1 Advanced Globbing

Zsh offers powerful pattern matching:

```
# Extended globbing
ls **/*.txt          # Recursively find all .txt files
ls *.{jpg,png}       # Files ending in .jpg or .png
ls file<1-100>.txt   # file1.txt through file100.txt

# Qualifiers
ls *(.)              # Only files
ls */                # Only directories
ls *(m-7)            # Modified in last 7 days
ls *(.L+1m)          # Files larger than 1MB
```

5.2 Directory Stack

```
# Navigate between directories
pushd /path/to/dir    # Push directory onto stack
popd                  # Pop and return to previous
dirs -v               # View directory stack
cd ~3                 # Jump to 3rd directory in stack
```

5.3 Tab Completion

Zsh has intelligent tab completion:

- Type partial command + Tab to complete
- Tab again to cycle through options
- Works for files, directories, commands, and arguments
- Case-insensitive by default

6 Process Management

Command	Description
<code>ps aux</code>	List all running processes
<code>top</code>	Interactive process viewer
<code>htop</code>	Enhanced process viewer (if installed)
<code>kill PID</code>	Terminate process by ID
<code>killall name</code>	Kill all processes by name
<code>command &</code>	Run command in background
<code>jobs</code>	List background jobs
<code>fg</code>	Bring background job to foreground
<code>Ctrl+Z</code>	Suspend current process
<code>bg</code>	Resume suspended job in background

7 Permissions and Ownership

```
# View permissions
ls -l file.txt           # Shows: -rw-r--r--

# Change permissions
chmod 755 script.sh      # rwxr-xr-x (executable)
chmod +x script.sh       # Add execute permission
chmod -w file.txt        # Remove write permission

# Change ownership
chown user:group file.txt # Change owner and group
sudo chown root file.txt  # Change owner to root
```

Permission notation:

- r (read) = 4
- w (write) = 2
- x (execute) = 1
- 755 = rwxr-xr-x (owner: all, group/others: read+execute)
- 644 = rw-r-r- (owner: read+write, others: read only)

8 Input/Output Redirection

```
# Redirection operators
command > file.txt      # Redirect output to file (overwrite)
command >> file.txt     # Append output to file
command < input.txt     # Read input from file
command 2> error.log    # Redirect errors to file
command &> all.log      # Redirect both output and errors

# Pipes
```

```
ls -l | grep ".txt"      # Pipe output to grep
cat file.txt | wc -l     # Count lines
history | tail -20       # Last 20 commands
```

9 Variables and Environment

```
# Variables
NAME="value"             # Set variable (no spaces!)
echo $NAME               # Use variable
export PATH=$PATH:/new/path # Add to PATH

# Common environment variables
echo $HOME               # Home directory
echo $USER               # Current username
echo $SHELL              # Current shell
echo $PATH               # Executable search path

# Zsh-specific
echo $ZSH_VERSION        # Zsh version
```

10 Aliases and Functions

10.1 Aliases

Add to ~/.zshrc file:

```
# Common aliases
alias ll='ls -lah'
alias ..='cd ..'
alias ...='cd ../../'
alias gs='git status'
alias gp='git push'
alias gc='git commit'

# After editing .zshrc, reload:
source ~/.zshrc
```

10.2 Functions

```
# Define function in .zshrc
mkcd() {
    mkdir -p "$1" && cd "$1"
}

# Usage:
mkcd new/nested/directory
```

11 History and Shortcuts

11.1 Command History

history	# Show command history
!!	# Repeat last command
!-2	# Repeat 2nd-to-last command
!123	# Repeat command #123
!grep	# Repeat last grep command
^old^new	# Replace 'old' with 'new' in last command

11.2 Keyboard Shortcuts

Shortcut	Action
Ctrl+A	Go to beginning of line
Ctrl+E	Go to end of line
Ctrl+U	Clear line before cursor
Ctrl+K	Clear line after cursor
Ctrl+W	Delete word before cursor
Ctrl+L	Clear screen
Ctrl+R	Reverse search history
Ctrl+C	Cancel current command
Ctrl+D	Exit shell / EOF
Tab	Auto-complete

12 Network and System

# Network	
ping google.com	# Test connectivity
curl https://example.com	# Fetch URL content
wget https://file.com/file	# Download file (if installed)
ifconfig	# Network interface info
netstat -an	# Network connections
# System information	
uname -a	# System information
df -h	# Disk usage
du -sh *	# Directory sizes
free -h	# Memory usage (Linux)
uptime	# System uptime
date	# Current date/time
# macOS specific	
sw_vers	# macOS version
system_profiler	# Detailed system info
open .	# Open current directory in Finder
open file.txt	# Open file with default app
pbcopy < file.txt	# Copy file contents to clipboard
pbpaste > file.txt	# Paste clipboard to file

13 Package Management

macOS uses Homebrew for package management:

```
# Install Homebrew (if not installed)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Common Homebrew commands
brew install package_name # Install package
brew uninstall package    # Remove package
brew update                # Update Homebrew
brew upgrade               # Upgrade all packages
brew list                  # List installed packages
brew search query          # Search for packages
brew info package          # Package information
```

14 Customizing Zsh

14.1 Configuration File

Your Zsh configuration lives in `~/.zshrc`. Edit with:

```
nano ~/.zshrc
# or
vim ~/.zshrc
```

After editing, reload:

```
source ~/.zshrc
```

14.2 Oh My Zsh (Optional)

Oh My Zsh is a popular framework for managing Zsh configuration:

```
# Install Oh My Zsh
sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"

# Configuration file becomes: ~/.zshrc
# Themes directory: ~/.oh-my-zsh/themes/
# Plugins directory: ~/.oh-my-zsh/plugins/
```

Popular plugins to add to `.zshrc`:

```
plugins=(git sudo z zsh-autosuggestions zsh-syntax-highlighting)
```


15 Quick Reference Table

Category	Command	Description
Navigation	<code>cd</code>	Change directory
	<code>pwd</code>	Print working directory
	<code>ls</code>	List contents
Files	<code>touch</code>	Create file
	<code>cp</code>	Copy
	<code>mv</code>	Move/rename
	<code>rm</code>	Remove
Viewing	<code>cat</code>	Display file
	<code>less</code>	Page through file
	<code>head/tail</code>	First/last lines
Search	<code>find</code>	Find files
	<code>grep</code>	Search in files
Help	<code>man command</code>	Manual for command
	<code>command --help</code>	Quick help
	<code>which command</code>	Find command location

16 Tips and Best Practices

1. Use **tab completion** — save time and avoid typos
2. Be careful with `rm -rf` — it deletes permanently
3. Use **man pages** — `man command` shows documentation
4. Test commands with `echo` — verify before execution
5. Use `Ctrl+R` — search command history interactively
6. Create **aliases** — for frequently used commands
7. Comment your **scripts** — use `#` for comments
8. Use **quotes** — for filenames with spaces: `"my file.txt"`

17 Getting Help

<code>man command</code>	<code># Full manual page</code>
<code>command --help</code>	<code># Quick help</code>
<code>command -h</code>	<code># Short help (sometimes)</code>
<code>which command</code>	<code># Find command location</code>
<code>whence -v command</code>	<code># Zsh: command type and location</code>
<code>type command</code>	<code># Show how command will be interpreted</code>

18 Conclusion

This primer covers the essential Zsh commands for macOS terminal use. For more advanced topics, explore:

- Shell scripting
- Regular expressions
- Process substitution
- Zsh parameter expansion
- Custom completion functions

Practice regularly, and the command line will become second nature!